

CS112: Monday 28 September, 12:15 to 1:20

Friday we closed on `typedef int Item;`. One line at the top of `Pair.h`, and every method below it says `Item`, so changing what the class holds is a one-line edit instead of a hunt through two files. Before that, all of week 3 again: classes, `const` methods, `operator<<`.

Today: a size nobody knows until the program runs. Every array so far had a number you chose while writing the code. Today the number arrives from a file, or from the user, and no value you write in those brackets is right. `new` moves the decision to where the answer exists, and hands back an **address** rather than an array. That is the split the rest of the course is built on: the object on the stack holds an address, the data sits on the heap.

PART

Daily quiz, `week04d1`

1. An array whose size you do not know
2. `new`, `delete`, and `nullptr`
3. Where it actually lives, measured not drawn
4. A class that owns an array
5. The index trap
6. Tomorrow

Typed, not shown. Slides 5 onward are Wednesday.

Part 1. The wall

```
int n;
cout << "How many numbers? ";
cin >> n;

int data[n];           // this is the line to stare at
data[0] = 42;
```

It compiles. That is the surprise. Then it recompiles on screen:

```
$ g++ -Wall -std=c++17 -pedantic vla.cpp
warning: ISO C++ forbids variable length array 'data' [-Wvla]
```

Ask what the compiler would have to know, at compile time, to lay that variable out, and wait for it: how big it is. `n` is not known then, and cannot be.

`char grid[64][128];` in a03, you chose that. Now a lap of telemetry has as many samples as the file turns out to have. Too small and the program breaks on the next circuit. Too large and you waste most of it every lap.

Part 2. `new`, `delete`, and `nullptr`

```
int n;  cin >> n;

int *data = new int[n];           // the size can be a
variable
for (int i = 0; i < n; ++i)
    data[i] = 0;

delete [] data;                  // give it back
data = nullptr;                  // and stop holding the
address
```

While typing: **`new` hands back an address**, which is why the variable is a pointer; **the size may be a variable**, which is the reason this week exists; **`delete []` for arrays, plain `delete` for one object**.

Then the part tomorrow's lab cannot be started without:

`nullptr` is C++ for "this pointer deliberately points at nothing", which is not the same as a pointer you never initialised. `delete[]` on `nullptr` is safe and does nothing; `delete[]` on garbage is a crash. A constructor that sets its pointer to `nullptr` has a destructor that is already correct, before you have written one.

Part 3. Where it actually lives

Do not draw the stack and the heap. Print them.

```
int onStack = 7;
int *heap = new int[3];
cout << "stack addr = " << (void*)&onStack << endl;
cout << "heap  addr = " << (void*)heap    << endl;
```

```
stack addr = 0x7ffc09a5f334
heap  addr = 0x55f5f217f270
```

Let them count the digits. The stack holds locals, is freed at end of scope, and is sized at compile time. The heap holds whatever `new` hands you, is freed only when **you** say so, and is sized while running.

Part 4. A class that owns an array

Start from the header and let them dictate the members.

```
typedef int Item;

class PyList {
public:
    PyList();                // empty
    PyList(int n);           // n items, all zero
    int getSize() const;
private:
    Item *myArray;           // the ADDRESS of an array on the
    heap
    int    mySize, myCapacity;
};
```

Ask before writing the body: **what should the empty one do with the pointer?** Take the answer from them. That is part 2 landing.

```
PyList::PyList() {
    myArray    = nullptr;
    mySize     = 0;
    myCapacity = 0;
}
```

Then the measurement that makes the design visible, and it takes 30 seconds:

```
PyList a;  PyList b(1000);
cout << sizeof(a) << " " << sizeof(b) << endl;    // 16 16
```

The object holding a thousand items is the same size as the empty one. The items are not in the object. If one sentence survives today, this is it.

Say the capacity line here and leave it alone: two counts, not one, how many it holds and how much room the array has. Wednesday answers why.

Part 5. The index trap

```
Item PyList::getIndex(int i) const {  
    if (i >= mySize) throw range_error("index out of range");  
    return myArray[i];  
}
```

Someone will ask why there is no `i < 0`. Answer by running it:

```
unsigned i = -1;  
cout << i << endl;                // 4294967295  
cout << (i >= mySize) << endl;    // 1, caught
```

An `unsigned` index can never be negative, which is a trap rather than a convenience. Pass `-1` and it arrives as 4294967295, so one test catches both a negative argument and a too-large one. With `-wextra` the compiler says the `i < 0` half is always false.

The bound is `mySize`, not `myCapacity`: the array may have room the container is not using, and those slots are not theirs.

Part 6. Tomorrow

Read `Trace.h` before you write anything. The whole class is on one page, and it opens with the `typedef` we ended Friday on.

Then run `make` and `make test` before you change a line. It builds, and almost every test fails. Both are correct. That is the starting point, so you can tell later what you moved.

Work in the order the file is written in. It is written in the order the pieces depend on each other, so out of order means debugging two things at once.